REOP 2021 - Session 2 (Shortest paths)

Guillaume Dalle

September 29, 2021

Contents

1 Statement and overview **1**
1.1 Shortest path problem . . . 1
1.2 Integer Programming formulation . . . 1
1.3 Complexity . . . 2
1.4 Dynamic programming . . . 3
1.5 Generalizations . . . 3
1.5.1 Shortest path variants . . . 3
1.5.2 Dynamic Programming for MDPs . . . 3

2 Solution algorithms **3**
2.1 Directed Acyclic Graphs (DAGs) . . . 3
2.1.1 Bellman principle . . . 3
2.1.2 Topological ordering . . . 4
2.2 Directed graphs with no absorbing cycles . . . 4
2.2.1 Bellman principle . . . 4
2.2.2 Bellman-Ford algorithm . . . 4
2.3 Directed graphs with nonnegative costs . . . 4
2.3.1 Dijkstra's algorithm . . . 4
2.3.2 A* algorithm . . . 5

1 Statement and overview

1.1 Shortest path problem

Input:

- a directed graph $D = (V, A)$
- a cost function $c : A \to \mathbb{Q}$
- two vertices o and d

Output: an $o \to d$ path P of minimum cost $c(P) = \sum_{a \in P} c(a)$ (or a proof that none exists because o and d are not connected)

We denote by $c(v)$ the cost of a shortest simple $o \to v$ path.

In cases where the problem is not well-defined (example: all weights are negative), we will ask for the shortest *simple* path instead.

1.2 Integer Programming formulation

Reminders

A Linear Program is an optimization problem with a linear objective and linear constraints:

$$\min_{x \in \mathbb{R}^n} c^\top x \quad \text{s.t.} \quad Ax \leq b \tag{LP}$$

A Mixed Integer Linear Program is a LP where some of the variables are constrained to be integers

$$\min_{x \in \mathbb{Z}^p \times \mathbb{R}^{n-p}} c^\top x \quad \text{s.t.} \quad Ax \leq b \tag{MILP}$$

In theory, solving a MILP is hard. However:

- they are very useful to model lots of real-life problems
- there are practically efficient solvers that can handle millions of variables if the problem has a certain structure

Formulation

Let x_{uv} be a binary variable equal to 1 if edge (u, v) is part of the path we choose, and 0 otherwise.

The shortest path problem can be stated as:

$$\min_x \sum_{(u,v) \in A} c(u, v) x_{uv} \quad \text{s.t.} \quad x \in \{0, 1\}^A \text{ defines an } o \to d \text{ path}$$

The constraint "x defines an $o \to d$ path" can be expressed linearly: if an vertex v different from o and d is visited, then it needs one incoming edge and one outgoing edge:

$$\forall v \in V, \quad \sum_{u \in \mathcal{N}^-(v)} x_{uv} - \sum_{w \in \mathcal{N}^+(v)} x_{vw} = \begin{cases} 0 & \text{if } v \neq o, d \\ -1 & \text{if } v = o \\ 1 & \text{if } v = d \end{cases}$$

1.3 Complexity

Theorem: The shortest paths problem is NP-complete in the general case.

Proof:

- Reduction from Hamiltonian path to longest simple path
- Reduction from longest simple path to shortest simple path

Polynomial cases:

- Unweighted graph: Breadth-First Search
- Acyclic graphs:
  - Undirected: forest (at most 1 path between each pair)
  - Directed: topological sorting
- Nonnegative costs
  - Directed & undirected: Dijkstra's algorithm
- No negative / absorbing cycles
  - Directed: Bellman-Ford
  - Undirected: T-joints

Algorithm	Naive complexity	Best complexity
Topological sorting	$O(m + n)$	$O(m + n)$
Dijkstra	$O(n^2)$	$O(m + n \log n)$
Bellman-Ford	$O(mn)$	$O(mn)$

1.4 Dynamic programming

Underlying idea: Bellman's optimality principle

> A subtrajectory of an optimal trajectory is itself optimal.

Generalize the problem to derive a recursion called the Bellman equation. Usually done by changing the bounds or adding parameters.

Knapsack example?

First compute the value of a solution. Is is usually easy to go back to the minimizer by working your way backwards.

1.5 Generalizations

1.5.1 Shortest path variants

Various flavors:

- Single source o, multiple destinations: the one studied here
- Single source o, single destination d: almost as hard
- Multiple sources, multiple destinations: harder

Extensions:

- Shortest paths with resource constraints (A. Parmentier's thesis)
- Multi-criteria shortest paths
- Shortest paths on timed transportation networks

1.5.2 Dynamic Programming for MDPs

See course notes

Talk about my internship at EDF on the optimization of cleaning schedules for photovoltaic solar panels

2 Solution algorithms

2.1 Directed Acyclic Graphs (DAGs)

2.1.1 Bellman principle

Proposition 5.4:

Let D be a DAG, P be an $o \to v$ path ending with edge (u, v), and Q be an $o \to u$ path. If P is a shortest $o \to v$ path, then Q is a shortest $o \to u$ path.

Proof:

- Suppose there is another $o \to u$ path Q' such that $c(Q') < c(Q)$.
- $P' = Q' \cup (u, v)$ is an $o \to v$ path.
- P' has a cost $c(P') = c(Q') + c(u, v) < c(Q) + c(u, v) = c(P)$.

Recursive equation

In a DAG, all paths are elementary and thus simple.

The length of a shortest simple $o \to v$ path satisfies the Bellman equation

$$c(v) = \min_u \in N^-(v) c(u) + c(u, v) \quad \text{and} \quad c(o) = 0$$

We can compute its values using *dynamic programming*. By remembering, for each v, the in-neighbor u that achieved the minimum, we can build a shortest-path tree.

Problem: in which order do we enumerate the vertices? The constraint is that we must compute $c(u)$ before $c(v)$ if there is an edge (u, v) in A.

2.1.2 Topological ordering

A digraph $D = (V, A)$ is acyclic iff there exists a total order $\preceq$ (i.e. a numbering of the vertices) such that $(u, v) \in A \implies u \preceq v$.

We define the operation $\texttt{DFS}(v)$ (Depth-First Search) as follows:

1. open v (put in S)
2. scan its children
3. close v (put in L)

This recursive definition is consistent since the graph has no cycles.

If we add a dummy vertex which has all the "orphan" nodes as children, we can consider the case with only one orphan node o. Then, Algorithm 2 is equivalent to applying $\texttt{DFS}(o)$.

Reversing the order in which vertices are closed yields a topological sort, since children are always closed before their parents.

2.2 Directed graphs with no absorbing cycles

2.2.1 Bellman principle

Proposition 5.2:

Let P be an $o \to v$ path with k arcs and Q be an $o \to u$ path, where u is the vertex before v on P. If P is a shortest $o \to v$ path among those with k arcs, then Q is a shortest $o \to u$ path among those with $k - 1$ arcs.

Proof:

- Suppose there is another $o \to u$ path Q' with $k - 1$ arcs such that $c(Q') < c(Q)$.
- $P' = Q' \cup (u, v)$ is an $o \to v$ path with k arcs.
- P' has a cost $c(P') = c(Q') + c(u, v) < c(Q) + c(u, v) = c(P)$.

2.2.2 Bellman-Ford algorithm

The length of a shortest $o \to v$ path satisfies the Bellman equation

$$c(v, k) = \min_u \in N^-(v) c(u, k - 1) + c(u, v)$$

with initial conditions

$$c(v, 0) = \begin{cases} 0 & \text{if } v = o \\ -\infty & \text{otherwise} \end{cases}$$

We can compute its values starting from $k = 0$. But when do we stop?

Solution: since D has no negative cycles, there is a simple shortest path of length at most $n - 1$. We compute $c(v, k)$ for all $v \in V$ and $k \in [0, n]$, and then pick k minimizing $c(d, k)$.

2.3 Directed graphs with nonnegative costs

2.3.1 Dijkstra's algorithm

Pseudocode:

Input: a digraph $D = (V, A)$ and costs $c \in \mathbb{Q}_+^A$.

1. Set $U = \emptyset$ (set of visited vertices)
2. Set $\lambda(v) = 0$ if $v = o$ and $\lambda(v) = +\infty$ otherwise (tentative distances)
3. While $V \backslash U \neq \emptyset$:
    1. Choose $v \in V \backslash U$ such that $\lambda(v) = \min_{v' \in V \backslash U} \lambda(v')$ (choose the closest unvisited vertex according to the tentative distance)
    2. Add v to U (visit it)
    3. Set $\lambda(w) = \min\{\lambda(w), \lambda(v) + c(v, w)\}$ for all $w \in N^+(w)$ (update the tentative distances of its neighbors)

Output: the vector λ which contains all distances $o \to v$

Proposition: (Values of the tentative distance)

- For all $u \in U$, $\lambda(u) = c(u)$
- For all $w \in V \backslash U$, $\lambda(w) = \min_{u \in U} c(u) + c(u, w) \geq c(w)$

Proof: Make a drawing!

Since they hold after initialization, we must only check that these properties are preserved by the loop.

Let v be the vertex achieving $\min_{v' \in V \backslash U} \lambda(v')$. By property 2, $\lambda(v) = \min_{u \in U} c(u) + c(u, v')$, so let u be the minimizer.

If $\lambda(v) > c(v)$ then there is an $o \to v$ path shorter than $\lambda(v)$. Let v' be its first vertex outside of U.

By choice of v, the red path is at least as long as the green path. By positivity of c, the black path has nonegative cost. Hence red + black is at least as long as green path.

Therefore, $\lambda(v) = c(v)$ and the first property is preserved. The second property is left as an exercise.

2.3.2 A* algorithm

Idea: speed up Dijkstra using a heuristic $h(v)$ to lower-bound the remaining distance $v \to d$.

Procedure: grow a set of paths and trim the ones that are hopeless.

Special case of Branch & Bound and LP duality.

Essential in transportation networks because the graphs are large but we have an idea of where to go.

Homework

Exercises 5.9, 5.12, 5.19, 5.20

References